

Presentation March 11th

Slide 1 | Introduction

Speaker Notes:

- Good evening everyone! Thank you for being here today.
- My name is Ian, and I have a background in Electrical Engineering from Christian Brothers University and a master's in computer science from Georgia Tech, specializing in Machine Learning.
- My research includes two published papers in deep learning alongside Dr. Hussin Ragb at Christian Brothers University. These papers were on convolutional neural network ensembles for malaria prognosis and covid diagnosis from chest x-ray images.
- Beyond my technical work, I'm passionate about mentorship and volunteer at Youth Villages. I'm also an active member of Second Presbyterian Church, which has been an important part of my life.
- I would like to bring attention to the QR code on most of the slides. If you scan this QR code, it will take you to the presentation website which will provide additional resources, code examples, and slide notes.
- Additionally, on the top of the website, you will find a Q/A form where you can write up a question and it will go directly to my email. If you have any questions during this presentation, please submit them there and I will discuss them during the end of this session. If the questions is difficult to answer, I may need to respond back to you after a few days, so don't forget to add your email in the form.
- Today, I'll be walking you through key technical concepts and best practices in data engineering, machine learning, and software testing. Let's get started!

Slide 2 | Data Pipeline Overview

Speaker Notes:

- This slide introduces the data pipeline, from raw data ingestion to dashboard updates.
- Snowflake retrieves production premiums and claims data, sending it to Databricks for processing.
- Snowflake is a cloud-based data platform designed for data warehousing, analytics, and data sharing. It operates on major cloud providers, including AWS, Azure, and Google Cloud, and is known for its scalability, performance, and ease of use.
- Databricks executes transformations such as filtering, trend calculations, and redundancy removal.
- Databricks is a unified data and AI platform built on Apache Spark, designed for big data processing, analytics, machine learning, and AI workloads. It provides a collaborative

environment that integrates data engineering, data science, and business analytics into a single platform.

- The processed data is stored in Snowflake in an append-only format, ensuring historical data integrity.
- GitHub plays a crucial role in regression testing and validation, while Streamlit updates dashboards for end users.
- This architecture ensures a robust and automated workflow for processing insurance and claims data.

Slide 3 | Medallion Architecture

Speaker Notes:

- The Medallion Architecture follows a layered approach to data processing, enhancing structure and reliability.
- Data flows through three layers:
 - **Bronze:** Raw ingestion from Snowflake.
 - **Silver:** Cleaned and structured data, stored in Parquet format.
 - **Gold:** Business-ready data, optimized for analysis and reporting.
- Databricks orchestrates these transformations, triggered via GitHub Actions for automation.
- This ensures a scalable, reproducible, and efficient data processing pipeline.

Slide 4 | Understanding Idempotency and Determinism

Speaker Notes:

- I use idempotency and determinism interchangeably here, but these are both useful and distinct properties of data pipelines. Determinism (as opposed to stochasticity) is important for testing, as testing requires that the outputs remain the same given the same inputs.
- Idempotency is a crucial concept in software and data engineering.
- It ensures that running a function multiple times with the same input always yields the same output.
- For example, adding an element to a set is idempotent (as the set will ignore updates if the repeat value is added), but appending to a list repeatedly is not.
- Idempotency is often achieved in a data pipeline by making sure that the outputs of a data engineering process do not lead to duplication in the resultant table. In our append-only data structure, we use a `run_date` which goes down to the millisecond level to ensure idempotency in write outputs. We then pull the latest “`run_date`” into subsequent pipelines.

- Idempotency prevents inconsistencies in data pipelines and is critical for reliable application testing.
- It is also worth pointing out that race conditions can cause non-idempotent pipelines. **Race Condition** are when multiple parallel operations execute at different times and can cause unexpected defects in the pipelines. Ideally, we would want data engineering pipelines that are immune to race conditions and thus are idempotent.

Slide 5 | Integration Snapshot Testing

Speaker Notes:

- Snapshot testing is widely used in analytics engineering.
- A parent function processes data using dependency injection (e.g., Parquet files).
- Child functions execute sequentially, and their output is compared to a saved expected result.
- If the results match, the test passes; otherwise, it fails, ensuring regression checks for updates.
- This approach helps validate data integrity over time.
- The snapshot test – however – does not conceptually validate the pipeline is correct. It only ensures that the pipeline output has not changed or “regressed”.

Slide 6 | Integration Unit Testing

Speaker Notes:

- Similar to snapshot testing but focuses on verifying output using an independent trusted source.
- The actual function’s output is compared against an expected output from a verified source like an LLM or manual calculation.
- If they match, the test passes, increasing confidence in correctness.
- This method is critical for ensuring software reliability, especially in high-stakes applications.
- This is pretty useful in analytics or accounting too, as a unit test is akin to reconciling numbers to a predetermined “ledger”.

Slide 7 | Alternative Testing Methods

Speaker Notes:

- The foundation of good software is robust testing.
- Four key testing methods:
 - **Smoke Testing:** Ensures the application runs without crashing. This is the simplest of tests and often the first one that gets run. You're probably doing smoke testing with your application without even knowing it. Smoke testing is what you're doing when you're trying to get your application to run while debugging it.
 - **Performance Testing:** Evaluates execution speed and efficiency. Performance testing is valuable from a user experience perspective, whether those users are developers or consumers using the product. A failed performance test often leads to performance refactors.
 - **UI Testing:** Checks for consistent and functional user interface behavior.
 - **Property Testing:** Verifies that outputs meet expected constraints. This is useful when you know data must uphold particular properties – such as a few specific columns values on a few different rows must add up to a particular value – are upheld by the eventual output. This is often done directly after an operation is performed on the data. You can think of this as a quality-assurance check.
 - **DevOps/Automation Testing:** This is how you would go about testing the developer workflow and the CI/CD process.
- Each of these plays a vital role in software quality assurance.

Slide 8 | Snapshot vs. Unit Testing

Speaker Notes:

- Snapshot and unit testing serve different purposes.
- **Snapshot Testing:**
 - Depends on previous outputs, requiring updates when logic changes.
 - Should be peer-reviewed to avoid false positives.
- **Unit Testing:**
 - Focuses on validating core functionality.
 - Helps developers build confidence in their code.
- A good balance between the two ensures robust software reliability.

Slide 9 | Test-Driven Development (TDD)

Speaker Notes:

- TDD is a methodology where tests are written **before** the code.

- The cycle follows:
 1. **Write a failing test**
 2. **Write code to pass the test**
 3. **Refactor and improve the code**
- This approach ensures better code quality and adherence to requirements.

Slide 10 | Async vs. Sync Processing

Speaker Notes:

- **Synchronous Processing** executes operations in sequence, leading to longer runtimes.
- **Asynchronous Processing** allows concurrent execution, improving efficiency.
- Asynchronous methods are commonly used in scalable data pipelines to reduce bottlenecks.

Slide 11 | Chunking vs. Parallelization

Speaker Notes:

- **Chunking** processes data sequentially with minimal memory overhead.
- **Parallelization** distributes tasks across multiple cores, reducing processing time.
- Parallelization is ideal for large-scale computations but requires careful memory management.

Slide 12 | Programming Language Trade-Offs

Speaker Notes:

- Different languages and frameworks have trade-offs:
 - **Pandas**: Great for small to medium datasets.
 - **Polars**: Optimized for efficiency and speed.
 - **PySpark**: Best for distributed big data processing.
 - **SQL**: Ideal for structured queries and cloud-based data operations.

Slide 13 | Pandas vs. Polars Performance

Speaker Notes:

- **Polars** outperforms **Pandas** in execution time for large datasets.

- It leverages parallel processing, making it more scalable.
- As dataset sizes increase, Polars maintains efficiency, making it ideal for performance-critical applications.

Slide 14 | Batch vs. Stream Processing

Speaker Notes:

- **Batch Processing** handles data at scheduled intervals, offering flexibility in tool selection.
- **Stream Processing** continuously processes data in real-time but requires specialized frameworks.
- The choice depends on system requirements and latency constraints.

Slide 15 | Vectorization in Python

Speaker Notes:

- Loop-based approaches are significantly slower in Python as these approaches rely on the Python interpreter.
- Vectorization optimizes performance by applying operations to entire arrays instead of individual elements.
- The goal of vectorization in Python is to write your code in such a way that it can be compiled and executed in a lower-level language such as C, Fortran, or Rust.

Slide 16 | CI/CD Pipeline

Speaker Notes:

- CI/CD automates the deployment process.
- Steps:
 1. Code is pushed to a feature branch.
 2. Unit tests and peer reviews validate changes.
 3. Deployment moves through development, pre-production, and finally production.

Slide 17 | SQL Injection Attacks

Speaker Notes:

- SQL Injection occurs when user input is concatenated into queries unsafely.

- Using **parameterized queries** prevents attackers from manipulating database queries.

Slide 18 | Internet Computer Protocol (ICP)

Speaker Notes:

Additional Innovations by Internet Computer Protocol (ICP):

- **Reverse Gas Model** – Users don't pay gas fees; developers prepay for execution via "cycles."
- **Direct Integration with Bitcoin & Ethereum** – Smart contracts can hold BTC and ETH natively without bridges.
- **Canister Smart Contracts** – More scalable than traditional smart contracts, running web-scale apps.

Condensed Summary:

ICP enhances blockchain scalability with **Chain-Key Cryptography** (efficient key management & cross-chain integration), **Deterministic Time Slicing** (breaks large computations into manageable slices), **BLS Cryptography** (unique, secure digital signatures), and **cheap storage** (optimized resource use). Additionally, ICP eliminates gas fees for users, directly integrates with BTC/ETH, and enables scalable smart contracts.

Slide 19 | Model Parallel Training Techniques

Speaker Notes:

- **DDP (Distributed Data Parallel)**: Replicates models across GPUs for efficient training.
- **Model Parallel**: Splits models across GPUs for large-scale architectures.
- **Tensor Parallel**: Distributes tensors for optimized computation.

Slide 20 | Zero Redundancy Optimization (ZeRO)

Speaker Notes:

The Zero Redundancy Optimizer (ZeRO) is a memory optimization technology developed by Microsoft to facilitate the training of large-scale deep learning models by minimizing memory redundancies across data-parallel processes.

ZeRO Optimization Stages:

1. **Stage 1**: Partitions optimizer states, reducing memory usage by up to 4 times compared to traditional data parallelism.
2. **Stage 2**: Adds gradient partitioning to optimizer state partitioning, achieving up to an 8-fold reduction in memory usage.
3. **Stage 3**: Further partitions model parameters, allowing memory reduction to scale linearly with the degree of data parallelism. For instance, partitioning across 64 GPUs can yield a 64-fold memory reduction.

By addressing memory bottlenecks in deep learning training, ZeRO enables the development of larger and more complex models without necessitating extensive code modifications, thereby advancing the capabilities of AI research and applications.

Slide 21 | Graviton Secure GPU Execution

Speaker Notes:

Graviton is an architecture developed by Microsoft to establish Trusted Execution Environments (TEEs) on Graphics Processing Units (GPUs). It enables applications to offload security- and performance-sensitive kernels and data to a GPU, executing them in isolation from other code running on the GPU and all software on the host, including the GPU device driver, operating system, and hypervisor.

Key Features of Graviton:

- **Isolation:** Ensures that kernels execute securely, isolated from other GPU code and host software, safeguarding against potential threats from compromised system components.
- **Remote Attestation:** Provides mechanisms to verify the integrity of the GPU's execution environment, allowing external parties to trust that computations are performed securely.
- **Minimal Hardware Modifications:** Achieves TEE functionality without altering existing CPU architectures, GPU cores, memory management units (MMUs), or memory controllers. The necessary changes are confined to peripheral components like the GPU's command processor, facilitating integration into current GPU designs with low hardware complexity.
- **Secure Runtime Extensions:** Introduces enhancements to the GPU runtime, enabling secure data transfers and kernel executions. These extensions ensure that data and code remain protected throughout their lifecycle on the GPU.

Microsoft's implementation of Graviton on NVIDIA GPUs demonstrated low performance overheads, ranging from 17% to 33%, with encryption and decryption of traffic to and from the GPU being the main sources of overhead.

By integrating Graviton, applications can leverage GPU acceleration for sensitive computations without compromising security, making it particularly beneficial for workloads that require both high performance and stringent security measures.

Slide 22 | Retrieval-Augmented Generation (RAG)

Speaker Notes:

This diagram illustrates the **Retrieval-Augmented Generation (RAG) process**, where an LLM enhances responses with relevant external documents. It consists of three main stages:

1. Document Preparation

- Raw documents undergo **preprocessing** and **vector embedding creation** for efficient retrieval.

2. Retrieval Process

- A **user query** is encoded into a vector representation.
- The system searches a **vector database** and retrieves the **top-k relevant documents**.

- These documents are **aggregated** and used to construct a prompt.

3. **Augmented Generation**

- The **LLM generates a response**, leveraging both retrieved context and the user query.

The **Pipeline Controller** orchestrates these stages, ensuring smooth workflow execution and logging status updates.

Slide 23 | Q&A & Closing Remarks

Speaker Notes:

- Thank you for your time!
- I'm happy to take any questions now.